Token Probability Distributions in Multi-Turn GPU Kernel Generation

Author: leo-grayrat **Affiliation:** Independent research **Date:** 2026-10-18

Abstract

CUDA Agent attributes an early reinforcement-learning collapse to a distribution mismatch associated with the rarity of CUDA in pretraining. Its explanation connects that rarity to token probabilities near 10^{-9}, which would amplify small differences between training and rollout probabilities. We examine the proposed connection through per-token measurements collected during multi-turn kernel generation.

The main experiment comprises 128 runs on four elementwise or reduction tasks and 40 runs on five KernelBench Level 2 operator chains, yielding 98,533 and 542,611 tokens, respectively. CUDA/Triton vocabulary has a low-probability rate of 0.26% under a strict vocabulary definition, compared with 0.84% for other tokens. In the operator-chain collection, the rate is 0.60% inside code blocks and 13.56% outside them; excluding comments reduces the code rate to 0.33%. The lowest observed code-token probability is approximately 2.2×10^{-6} across 322,375 code tokens. Contextual inspection places the main contributions to the low-probability tail in comments, identifier naming, punctuation and malformed continuations. The inspected examples provide no clear instance of uncertainty between alternative kernel implementations.

These observations locate the probability cost primarily in the surrounding prose and surface form of the generated output. Failure-following responses account for 87.5% of the operator-chain tokens, making the composition of the interaction an important part of this distribution. A supplementary audit recovers 40 code-bearing tool arguments with probabilities for 3,162 generated tokens, demonstrating collection through the local tool-call interface alongside the reply-body route used in the main experiment.

Keywords: CUDA Agent; training instability; conditional probability; log-probability distribution; fusion kernels

1. Introduction

1.1. Training instability and the vocabulary explanation

Multi-turn reinforcement learning has enabled language models to develop GPU kernels through repeated implementation, evaluation and revision, as demonstrated by Kevin and CUDA Agent [1, 2]. In its main KernelBench table, CUDA Agent reports an overall pass rate of 98.8% and a faster-than-`torch.compile` rate of 96.8% across 250 workloads [2, 3]. Its training procedure uses proximal policy optimisation (PPO) over trajectories produced in a development sandbox [2, 4].

The authors report that their first run collapsed after 17 training steps. They attribute this instability to a severe domain distribution mismatch, citing a CUDA share below 0.01% of pretraining data and token probabilities near the numerical floor [2]. The illustrative probability in their explanation is

$$\pi_\theta(a_t \mid s_t) \approx 10^{-9}.$$

Here the action is the generated token and the state is its preceding context. At such a small denominator, a difference between the training and rollout probabilities could produce a large importance ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

The proposed remedy combines single-turn reinforcement learning, rejection fine-tuning on positive-reward trajectories and value pretraining of the critic. The explanation therefore links the composition of the pretraining corpus to a particular feature of the generated-token distribution, and then connects that feature to training instability.

1.2. Token-level measurement of the proposed mechanism

The proposed mechanism contains four steps: CUDA is rare in pretraining; that rarity produces near-floor token probabilities; those probabilities amplify precision differences in the importance ratio; and the resulting instability causes training to collapse. Each step concerns a different observable and calls for a corresponding measurement.

The second step can be examined by collecting conditional token probabilities during generation. This makes it accessible through an inference interface and allows the vocabulary explanation to be tested before undertaking a training run. We measure where low-probability tokens occur, inspect the code surrounding them, and compare the observed tail with the magnitude quoted by CUDA Agent. These measurements provide an empirical check on the distributional premise of the proposed mechanism.

1.3. Measurement constraints

The DeepSeek API responses inspected during setup returned probabilities for reply text and reasoning, while code written inside tool-call arguments lay outside those returned fields. For the main experiment, we therefore asked the model to place its candidate implementation in the reply body. A driver wrote the file, ran the evaluator and returned feedback for the next turn.

The saved local vLLM records expose a complementary interface. With Qwen3-4B-AWQ, the generated probability stream includes serialised tool calls and the code strings inside their arguments. Concatenating this stream in generation order makes it possible to recover the code and associate its spans with the recorded probabilities. Section 4.8 reports the evidence obtained through this route.

1.4. Design of this study

The main experiment measures conditional token probabilities in a multi-turn kernel-writing loop using two task families. Family A contains four elementwise or reduction tasks, and Family B contains five KernelBench Level 2 operator chains. The analysis addresses four questions:

- **RQ1:** How does the low-probability rate of CUDA/Triton vocabulary compare with that of other tokens?
- **RQ2:** How does the distribution differ between fenced code and surrounding prose?
- **RQ3:** How does the code-side rate vary across the selected reduction and operator-chain tasks?

- **RQ4:** How close are the observed token probabilities to the quoted floor?

We also inspect low-probability tokens in their original contexts to determine whether they occur at implementation decisions, identifier choices, comments or other parts of the output.

1.5. Summary of results

The vocabulary comparison places fewer low-probability tokens in the CUDA/Triton group than in the remainder of the output. The regional comparison shows a much larger separation between code and prose, which becomes more pronounced when comments are removed from the code region. Across the selected tasks, code-side rates remain low, and the measured code-token minimum stays above the quoted floor. The table summarises these observations; the following sections give the definitions and contextual evidence behind them.

Question	Observation
RQ1	Strict-vocabulary rates are 0.26% for CUDA/Triton and 0.84% for other tokens.
RQ2	Rates are 0.60% inside code fences and 13.56% outside them.
RQ3	Rates are 0.67% for reductions and 0.60% for operator chains, using the same fence rule.
RQ4	The lowest code-token probability is approximately 2.2×10^{-6}.

1.6. Scope

The main collections describe the inference-time output of an aligned commercial model, while CUDA Agent's explanation concerns a base model during training rollout. The comparison examines whether the proposed vocabulary effect appears in the measured output distribution. Section 5.2 discusses how model state enters the interpretation, and the local records extend the collection method to code generated through tool calls.

2. Related Work

2.1. The causal chain under test

CUDA Agent connects domain rarity, near-floor token probabilities and training instability, then uses staged warm-up to stabilise PPO [2]. Rejection fine-tuning retains positive-reward trajectories and removes unsuccessful or malformed interactions. Value pretraining uses generalised advantage estimation [5], and subsequent PPO training uses asymmetric clipping [2, 4]. The settings reported for these components are collected below [2]:

Component	Parameter	Value
Advantage estimation	Discount factor	1
Advantage estimation	Trace-decay factor	0.95
PPO clipping	Lower bound parameter	0.2
PPO clipping	Upper bound parameter	0.28

CUDA Agent's own ablations show reward collapse and rising policy entropy when rejection fine-tuning is removed, and lower explained variance with longer trajectories when value pretraining

is removed [2]. These observations describe training behaviour at the distribution and trajectory levels. The near-floor probability proposed in the same paper concerns an individual generated token, motivating a token-level measurement alongside the reported aggregate indicators [2].

2.2. Train/inference mismatch in DR. KERNEL

DR. KERNEL studies reinforcement learning for Triton kernel generation and examines both training instability from train/inference mismatch and an objective that can be misaligned with meaningful speedup [6]. It monitors entropy, gradient norms and perplexity, and applies geometric Mismatch Rejection Sampling to restrict the discrepancy between training and rollout probabilities.

For a trajectory with token index set T, the geometric-mean importance ratio is

$$w = \exp\left(\frac{1}{|T|}\sum_{t\in T}\log\frac{\pi_{\text{train}}(a_t \mid s_t)}{\pi_{\text{rollout}}(a_t \mid s_t)}\right).$$

The acceptance interval is

$$0.999 \le w \le 1.001.$$

A further token-level veto rejects any sequence containing a likelihood ratio below the specified threshold:

$$\frac{\pi_{\text{train}}(a_t \mid s_t)}{\pi_{\text{rollout}}(a_t \mid s_t)} < 10^{-4}.$$

DR. KERNEL reports that this procedure stabilises training dynamics without fundamentally raising the performance ceiling [6]. The result makes train/inference discrepancy an important comparison for the vocabulary account: the two explanations locate the relevant distributional difference in different parts of the generation and training process.

2.3. Precision mismatch and off-policy drift

Jet-RL studies the numerical discrepancy introduced when rollout uses FP8 precision and training uses BF16, and proposes a unified precision flow to stabilise optimisation [7]. Zheng et al. derive a first-order formulation of token-level policy-gradient objectives and relate its validity to training–inference discrepancy and policy staleness [8]. Yao et al. demonstrate that separate rollout and training backends can assign different token probabilities even with shared model parameters, motivating importance-sampling correction [9].

Trust Region Masking extends this analysis to long-horizon reinforcement learning by examining the sequence-level consequences of implementation divergence and masking trajectories outside a trust region [10]. Together, these studies motivate measuring the probabilities that enter the importance ratio and identifying where their tails occur [7, 8, 10]. Our experiment supplies the generated-token distribution for selected kernel-writing tasks, including its regional composition and contextual examples.

2.4. Logprob availability in closed APIs

Probability outputs are useful scientific observations in their own right. Chauvin et al. use token log-probabilities to detect changes in models served through APIs, showing that these measurements can support low-cost monitoring [11]. Carlini et al. recover information about a production model's output projection through black-box API access, illustrating the additional information exposed by probability-related outputs [12].

The serving interface also determines which parts of a generated interaction can be measured. Michaelov et al. distinguish model evaluation, comparison and interpretation as research goals with different access requirements [13]. In our DeepSeek experiments, moving code into reply text made its probabilities accessible. In the local vLLM records, the generated stream retained both tool-call arguments and their probabilities. Recovering the latter requires parsing the serialised output and locating the code within its JSON strings. These collection routes make the measured output channel explicit for each experiment.

2.5. Position of this work

The present study examines the generation-time distribution implicated by the vocabulary explanation. Its contribution is a measurement across two task families, supported by regional comparisons and contextual reading of the tail. The accompanying local audit establishes how code probabilities can also be recovered from tool arguments, extending the observable interface available for such experiments.

3. Method

3.1. Collection without an agent framework

The main experiment uses a driver to collect code directly from the reply body. During setup, an instruction to print code before issuing a tool call produced no compliant response in six trials. Moving file writing and execution into the driver allowed the model's implementation to remain within the API's returned probability fields throughout collection.

For each task, the driver sends a description and `reference.py`, and requests a complete `candidate.py` inside a fenced block. It extracts the block from `message.content`, writes the candidate, invokes an independent evaluator and appends the evaluation output to the conversation. A single-turn probe returned a 1,396-character code block containing `@triton.jit` and `tl.load`; all 448 tokens within the block had a recorded log-probability. The local collection uses the tool-call interface described in Section 3.5.

3.2. Generation and evaluation loop

At each turn, the driver submits the accumulated message history and records the model's response together with its token probabilities. The API request uses the following generation settings:

Setting	Value
`logprobs`	`true`
`top_logprobs`	20
`temperature`	1.0

Let M_t denote the message history at turn t, and let y_t denote the generated response. For token position i, the recorded value is

$$\ell_{t,i} = \log \pi_\theta(y_{t,i} \mid M_t, y_{t,<i}) .$$

The driver extracts the candidate block and stages it with the reference implementation and evaluator in an isolated temporary directory. After running the evaluator, it appends the response and evaluation feedback f_t to the history:

$$M_{t+1} = M_t \oplus [\text{assistant} : y_t] \oplus [\text{user} : f_t].$$

Here the concatenation operator appends messages in order. A run stops when a candidate passes after at least one revision, when the response contains no fenced block, or when the turn limit is reached. The limit is eight turns for the reduction tasks and six turns for the operator chains.

3.3. Two task families

Family A contains `row_reduce`, `vector_affine`, `layernorm_rows` and `softmax_rows`. These tasks use elementwise operations or single-dimension reductions, with short kernel bodies and limited control flow. Family B draws five operator chains from KernelBench Level 2, the operator-sequence split of a benchmark containing 250 PyTorch workloads [3]:

Task	Chain
`12_gemm_multiply_leakyrelu`	Linear → scale → LeakyReLU
`1_conv2d_relu_biasadd`	Conv2d → ReLU → bias
`14_gemm_divide_sum_scaling`	Gemm → divide → sum → scale
`17_conv2d_instancenorm_divide`	Conv2d → InstanceNorm → divide
`19_convtranspose2d_gelu_groupnorm`	ConvTranspose2d → GELU → GroupNorm

The operator chains offer several implementation arrangements:

- A candidate can fuse the chain into one kernel.
- It can fuse an elementwise tail into the epilogue of a heavy operation.
- It can retain separate operations with intermediate tensors.

These arrangements introduce choices about fusion, intermediate storage and tiling that broaden the implementation space represented in the experiment. The script `scripts/make_kb2_tasks.py` adapts each KernelBench `Model` class to the `run(x)` entry point. It seeds the model so that the reference and candidate use identical weights and exports the model as `MODEL` for the candidate to import. Evaluators check correctness on two shapes with a tolerance of 10^{-4}.

3.4. Feedback conditions

The experiment uses two feedback conditions to vary the amount of evaluator output returned to the model:

- **Full feedback** returns the evaluator's verbatim output.
- **Truncated feedback** returns one line containing `compilation failed`, `output mismatch` or `correct`.

Both procedures were implemented before collection and remained fixed throughout the experiment. The condition therefore changes the returned feedback while keeping the collection procedure consistent within each task family.

3.5. Model selection

Model selection was tied to the available collection interface and hardware. The main task families use a larger model through the DeepSeek API, with code placed in reply text. The earlier local experiments use Qwen3-4B-AWQ under vLLM on an 8 GB GPU and retain code in tool-call arguments.

In the local records, `content_logprobs` contains the generated tokens of serialised tool calls, including proposed file contents. The corresponding OpenCode events retain the tool names, input strings and execution status. Recovering a code argument involves concatenating the token stream, parsing the tool-call JSON and locating the `content` or `newString` value. Some responses have empty `content_text` and `tool_call_text` convenience fields even though their generated stream contains the code. Section 4.8 reports the audit of these saved records.

Families A and B use the non-reasoning API model with a maximum response budget of 8,192 tokens. The reasoning variant tested during setup exhausted a 4,000-token budget in the reasoning channel and returned an empty content field with `finish_reason=length`. The selected interface made reply-body code probabilities accessible while allowing a larger model than the local GPU could serve. This choice treats the API as the measurement instrument and specifies its observable output channel, following the distinction between model behaviour and deployment access discussed by Michaelov et al. [13].

An earlier vocabulary comparison from the local collection was withdrawn after a classification error was identified. Section 5.5 describes that error; the retained records support direct inspection of the generated code and its probabilities.

3.6. Observables

The analysis describes both the frequency and magnitude of low-probability tokens. For each region or token group, it reports the following quantities:

1. The fraction of tokens below a fixed log-probability threshold.
2. The 0.1st, 1st, 5th and 50th percentiles of the log-probability distribution.
3. The minimum log-probability and its corresponding absolute probability.
4. The fraction of total negative log-probability contributed by the lowest-probability 0.1%, 1% and 5% of tokens.

For a token set G, the threshold rate is defined as

$$R_\tau(G) = \frac{1}{|G|} \sum_{i \in G} \mathbf{1}[\log \pi_\theta(a_i \mid s_i) \leq \tau], \qquad \tau = -2.$$

The quantiles and minimum describe how far the tail extends, while the concentration statistic in Section 4.5 describes how much of the total cost it carries. Region assignment follows fenced-block state: tokens following an odd number of opening or closing triple-backtick delimiters are assigned to code, and the remaining tokens to prose. This regional analysis uses the generated text's structure without a vocabulary classifier.

3.7. Statistics

The vocabulary comparisons use a two-sided Fisher exact test on a two-by-two table whose dimensions are group membership and whether the token falls below the threshold. An exact test accommodates the unbalanced cell counts in these comparisons. Tokens within a completion are dependent, however, so token-level tests can overstate the precision of the reported differences. We report their effect sizes alongside the nominal significance values; the analysis does not include a run-level cluster adjustment.

3.8. Collection scale

The main experiment produced the following collections:

Family	Runs	Tokens
A (reductions)	128	98,533
B (chains)	40	542,611

Seven of the 40 Family B runs passed evaluation. Responses following failures account for 87.5% of that family's generated tokens, so the regional distributions largely describe output produced during unsuccessful revision. Section 5.6 gives the failure breakdown and identifies the contribution of the collection driver.

4. Results

4.1. RQ1: vocabulary-level distribution

In Family A, CUDA/Triton vocabulary has a lower threshold rate than the remaining tokens under both vocabulary definitions. The strict definition yields rates of 0.26% and 0.84%, while the loose definition yields 0.49% and 0.84%:

Boundary	Group	n	mean logprob	R_{-2}	Fisher p
strict	CUDA/Triton	2,344	−0.013	**0.26%**	**0.0007**
strict	other	96,189	−0.062	0.84%	
loose	CUDA/Triton	4,524	−0.038	**0.49%**	**0.0070**
loose	other	94,009	−0.062	0.84%	

A separate keyword-based structural group, comprising `for`, `if`, `def`, `return`, `in` and `not`, has a rate of 1.54%, compared with 0.81% for other tokens. Inspection of its low-probability examples reveals substantial contamination by prose. Of the 40 low-probability structural matches, 16 are ordinary English uses, including phrases such as `If you want to push speed further` and `Key design points for why`. The structural keyword count therefore combines programming syntax with natural-language occurrences. Section 4.7 examines the tokens in context to distinguish these uses.

Figure 1 compares the two vocabulary definitions and shows the strict groups' empirical cumulative distributions. The tail view makes the difference visible across probability levels, complementing the threshold counts above.

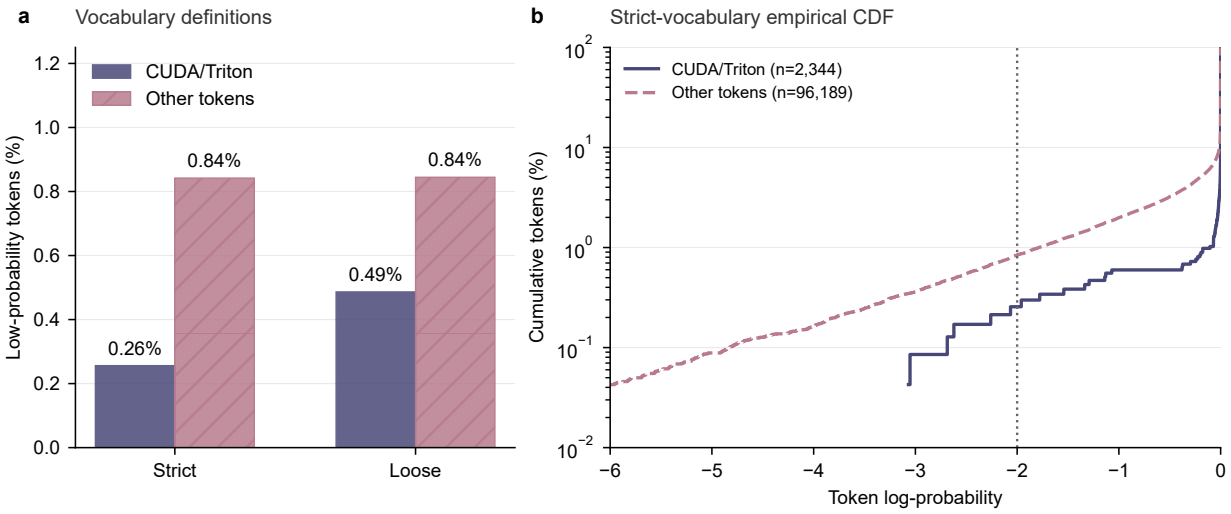


Figure 1: Vocabulary-level token distributions in Family A (98,533 tokens). Panel a shows the percentage of tokens with log-probability at or below −2 under strict and loose definitions. Panel b shows the empirical cumulative percentage for the strict groups on a logarithmic vertical scale; the dotted line marks the threshold.

4.2. RQ2: spatial distribution of the probability cost

The Family B collection contains 322,375 tokens inside fenced blocks and 220,236 outside them. Across all five tasks, the low-probability rate is higher outside the fenced region. The aggregate rates are 0.60% for code and 13.56% for prose, corresponding to a density ratio of 22.4:

Task	code tokens	code R_{-2}	prose tokens	prose R_{-2}
`12_gemm_multiply_leakyrelu`	55,419	**0.25%**	25,717	12.62%
`14_gemm_divide_sum_scaling`	46,002	**0.93%**	49,724	14.26%
`17_conv2d_instancenorm_divide`	77,984	**0.58%**	34,586	13.49%
`19_convtranspose2d_gelu_groupnorm`	89,908	**0.73%**	64,942	14.48%
`1_conv2d_relu_biasadd`	53,062	**0.52%**	45,267	12.03%
total	**322,375**	**0.60%**	**220,236**	**13.56%**

Comments contribute to the tail within code fences. Removing in-block comments lowers the aggregate code-side rate to 0.33%, with a decrease in every task:

Task	all code	pure code
`12_gemm_multiply_leakyrelu`	0.25%	**0.18%**
`14_gemm_divide_sum_scaling`	0.93%	**0.44%**
`17_conv2d_instancenorm_divide`	0.58%	**0.26%**
`19_convtranspose2d_gelu_groupnorm`	0.73%	**0.42%**
`1_conv2d_relu_biasadd`	0.52%	**0.37%**
total (310,935 tokens)	0.60%	**0.33%**

After this exclusion, the prose rate is approximately 41 times the code rate. The fence-based figure of 0.60% retains the same region rule used for Family A's 0.67%, allowing the two collections to be compared under that definition. In the complete Family B collection, frequent low-probability

tokens include `the` (118 occurrences), the backtick (116), the comma (70), the full stop (64), `and` (63) and `I` (55), reflecting the contribution of prose function words and punctuation.

The regional separation is visible in each task in Figure 2. Its cumulative distributions also show how the difference extends beyond the single threshold used in the table, with the prose tail reaching lower probabilities than the code tail.

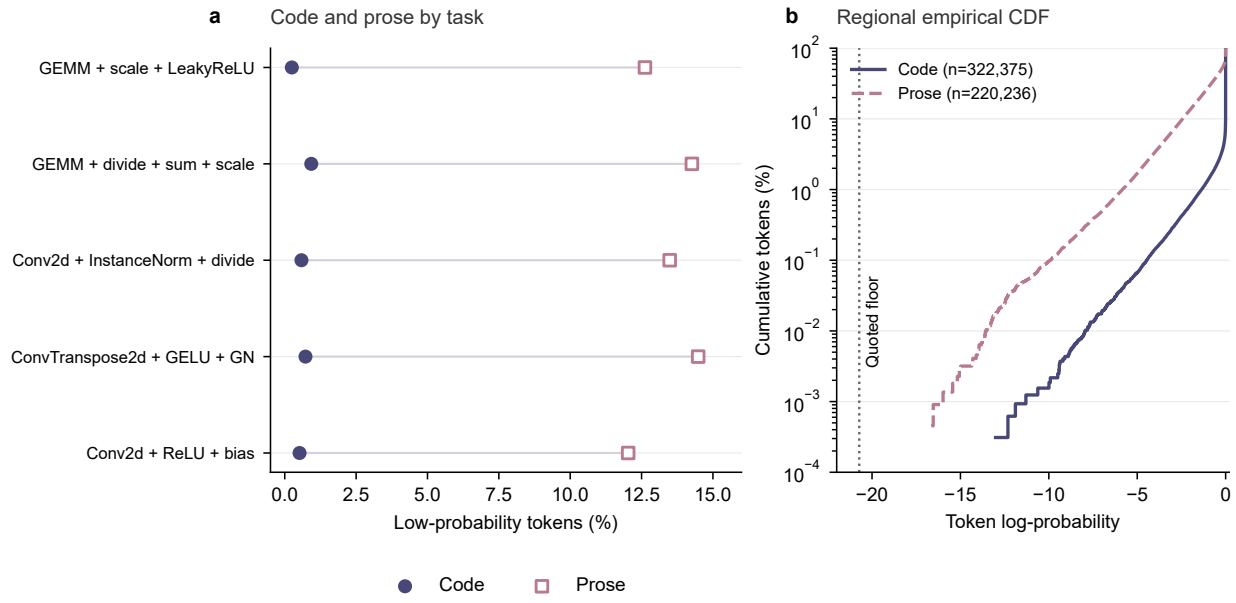


Figure 2: Regional distributions in Family B. Panel a connects the code and prose threshold rates within each task. Panel b plots their empirical cumulative percentages on a logarithmic vertical scale, with the quoted probability floor marked by a dotted line. Fence-based assignment includes comments within the code region.

4.3. RQ3: effect of task difficulty

The two task families differ in task construction, turn limit, success rate and context composition. Their aggregate code-side rates are 0.67% for reductions and 0.60% for operator chains under the same fence rule. The operator-chain rate falls to 0.33% after comments are excluded, although that value uses a different boundary from the reduction figure.

Within Family B, pure-code rates range from 0.18% to 0.44% across tasks with three or four operations and success counts ranging from zero to five of eight runs:

Task	chain	success	pure-code R_{-2}
`12_gemm_multiply_leakyrelu`	3 ops	5/8	0.18%
`1_conv2d_relu_biasadd`	3 ops	0/8	0.37%
`14_gemm_divide_sum_scaling`	4 ops	1/8	0.44%
`17_conv2d_instancenorm_divide`	3 ops	1/8	0.26%
`19_convtranspose2d_gelu_groupnorm`	3 ops	0/8	0.42%

The table shows no monotonic ordering of the code-side rate by chain length or success count. The selected operator chains retain a sparse code-side tail even when evaluations frequently fail. A more

substantial difference between the families appears in the surrounding prose: it accounts for 40.6% of Family B's stream and has task-level threshold rates between approximately 12% and 14%.

4.4. RQ4: proximity to the claimed probability floor

The regional quantiles show that most code tokens have probabilities close to one, while the prose distribution extends further into the low-probability tail. The measured values for Family B are

quantile	code	prose
p0.1	−4.460	−9.915
p1.0	−1.361	−5.842
p5.0	−0.063	−3.433
p50.0	−0.000	−0.240
min	**−13.016**	**−16.647**

Exponentiating the regional minima gives their absolute probabilities. The corresponding values, together with the log-probability of the quoted floor, are

$$\begin{aligned} \min p_{\text{code}} &\approx e^{-13.016} \approx 2.2 \times 10^{-6}, \\ \min p_{\text{prose}} &\approx e^{-16.647} \approx 5.9 \times 10^{-8}, \\ \log(10^{-9}) &\approx -20.7. \end{aligned}$$

The minimum across 322,375 code tokens is approximately four hundred times the quoted floor. The code distribution's first percentile corresponds to a probability of about 0.26, and its median rounds to one.

Whole-response probabilities operate on a different scale because they multiply the conditional probabilities of every token. Across 215 completions, joint log-probability has a mean of −903.3 and a median of −414.8. The probability corresponding to that median is

$$P_{\text{completion, median}} \approx e^{-414.8} \approx 7 \times 10^{-181}.$$

This cumulative quantity describes an entire response, whereas the floor comparison concerns a single conditional token probability.

4.5. Concentration of negative log-probability

To measure how much of a region's cost comes from its tail, we sum the negative log-probabilities of its tokens:

$$C(G) = -\sum_{i \in G} \log \pi_\theta(a_i \mid s_i).$$

After sorting tokens by increasing log-probability, let G_q contain the lowest-probability fraction q of the region. The concentration statistic is

$$\kappa_q(G) = \frac{C(G_q)}{C(G)}.$$

A value of 0.981 at the 5% cutoff means that those tokens contribute 98.1% of the region's total negative log-probability. Applying the statistic to Family B gives

region	tokens	Σ logprob	worst 0.1%	worst 1%	worst 5%
code	322,375	−14,276	13.4%	60.9%	**98.1%**
prose	220,236	−179,942	1.4%	9.3%	**30.3%**

Code has a sharply concentrated cost distribution: the lowest-probability 5% contributes 98.1% of its total. Prose has a broader distribution, with the corresponding subset contributing 30.3%. Its total negative log-probability is 12.6 times the code total despite containing only about 68% as many tokens, giving a per-token cost approximately 18 times higher.

Figure 3 places the cost concentration and regional minima together. The early rise of the code curve shows how a small fraction of tokens contributes most of its negative log-probability, while the prose curve accumulates that cost more gradually.

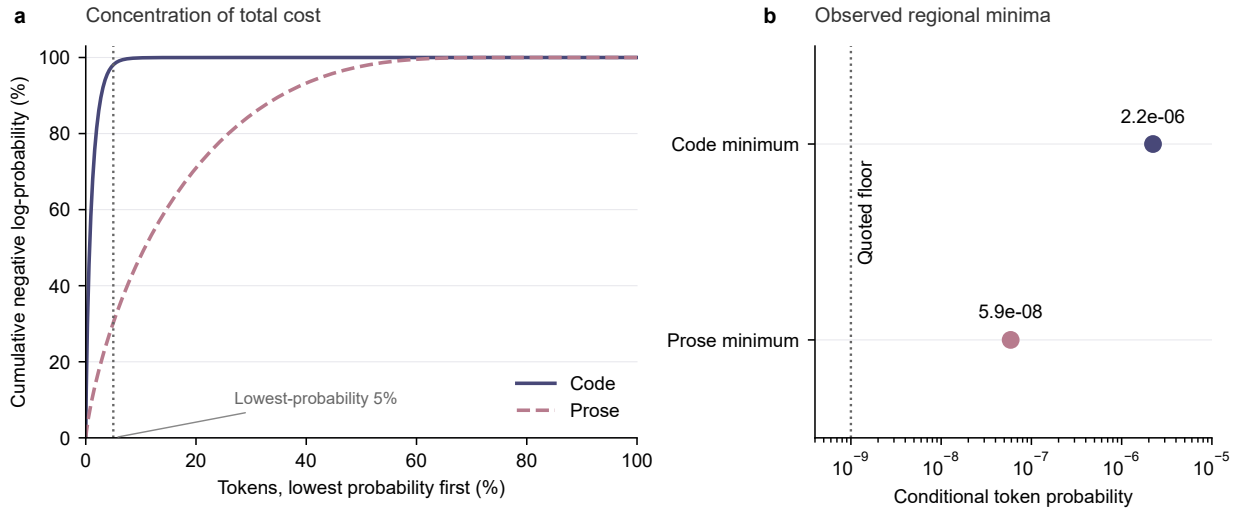


Figure 3: Tail magnitude and concentration in Family B (322,375 code tokens and 220,236 prose tokens). Panel a accumulates each region's negative log-probability after sorting tokens from lowest to highest probability; the dotted line marks the first 5% of tokens. Panel b shows the observed regional minimum probabilities on a logarithmic horizontal axis, alongside the quoted floor.

4.6. Context type

Responses are grouped by the evaluation context preceding them. In Family B, the threshold rate increases from 1.88% in initial responses to 4.26% after success and 6.37% after failure:

context	tokens	R_{-2}
initial	55,736	1.88%
after_success	12,147	4.26%
after_failure	474,728	**6.37%**

Failure-following responses have the highest aggregate rate and contribute 87.5% of the tokens. Family A also shows an increase after failure, with a rate 4.6 times the initial-response rate. The

two collections thus associate failure-following contexts with a larger low-probability tail. Their differing proportions of initial, successful and unsuccessful interactions also affect any comparison of aggregate rates.

Figure 4 displays the context mixture alongside the corresponding threshold rates. Showing these quantities separately makes the dominance of failure-following output visible without conflating its frequency with its per-token rate.

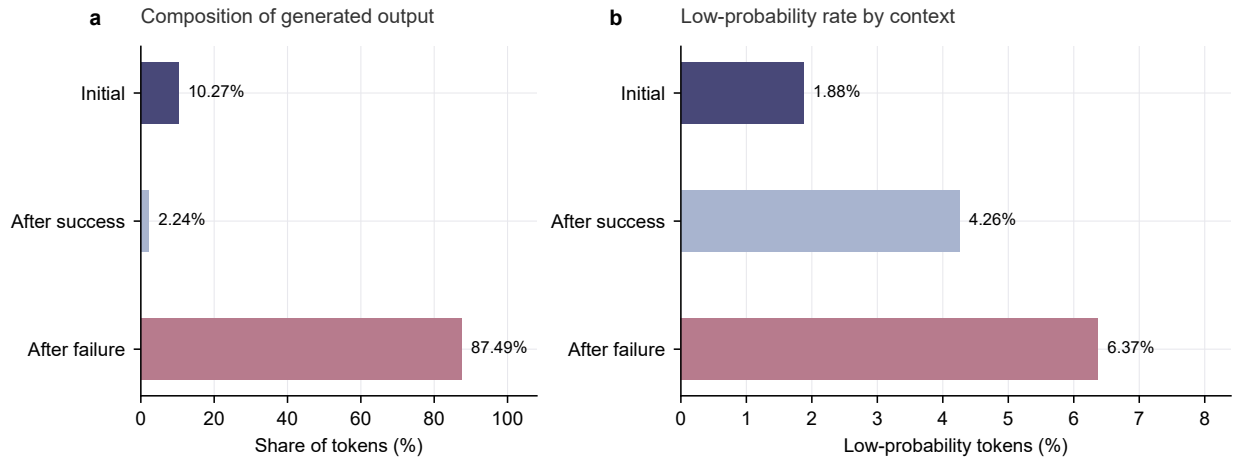


Figure 4: Evaluation-context composition and threshold rates in Family B (542,611 tokens). Panel a gives each context's share of all generated tokens. Panel b gives the percentage of low-probability tokens within that context. Categories and probabilities are computed from the saved response records using the same grouping as the reported table.

4.7. Manual classification of low-probability tokens

We inspected low-probability tokens in their original text to identify the activity taking place at each position. For Family A, this inspection covers all 816 tokens below the threshold of −2. The approximate category shares and examples are

category	share	examples
code comments / docstrings	~75%	`backed`, `moderate`, `overflow`, `Tunables`, `divisible`
identifier naming	~12%	`y`, `acc`, `denom`, `row_ptr`, `orig_shape`
punctuation / whitespace	~10%	`\n\n`, `:\n`, `,`, `(`
conversational prose	~4%	`If`, `Odd`, `Summary`, `verified`
control-flow decisions	**~1%**	`if not x.is_contiguous()`, `for col_start in range(...)`

For Family B, inspection covers 189 non-comment code tokens with log-probability at or below −4.5. The approximate counts are recorded below:

category	count	examples
identifiers (variables, params, function names)	~110	`mean_ptr`, `coff`, `KHW`, `k_rem`, `dots`, `gemv_kernel`
punctuation / whitespace	~40	`)`, `\n`, `,`, `:`
comment residue (missed by the `#` rule)	~15	`where y_in = y_out - ky`, `← NO`, `lsb-for-tie`
numbers / constants	~8	`2048`, `512`, `32`
method calls	~8	`.view`, `.to`, `.where`
operators	~6	`+=`, `-`, `*`
syntactic choices	**~5**	`for kh in static:`, `for i in tl.static_range(0,1):` `pass`, `assert ref is not None`

The inspected contexts provide no clear example of a low-probability token selecting between alternative kernel implementations, such as changing the fusion arrangement or choosing whether to retain an intermediate in registers. Most examples concern names, punctuation, residual comments or incomplete syntax.

The lowest code token is `//`, with log-probability −13.02, in `tl.arange(0, BLOCK // 1)`. The lowest prose token is `|=`, with log-probability −16.65, in the malformed string `allow_tf32|=|True`. Other tail examples include fragments in `biasp_ptr`, `rtne` and `tirual`, together with punctuation in the unreplaced placeholder `atol=?, rtol=?`. Their surrounding text identifies malformed or freely variable surface forms at these positions.

Identifier choices illustrate the distinction between surface variation and implementation. The name `y` in `y = torch.empty_like(x)` is repeatedly low-probability, although the same allocation can be written with `out` as the output name. The uncertainty recorded at that token therefore concerns the identifier used to express the allocation. Reading the surrounding statement makes this distinction visible in a way that a vocabulary count alone cannot provide.

4.8. Local tool-call capture

The saved local vLLM records retain code generated inside file-writing tool arguments together with its token probabilities. An audit of `runs`, `runs-triton` and `runs-migrated` covers 195 responses and 26,529 generated tokens. Parsing the reconstructed output identifies 40 code-bearing arguments across 33 responses, containing 3,162 generated tokens with finite log-probabilities. Every recovered code string matches the input of a corresponding `write` or `edit` event.

The audit identifies a code-bearing argument as a `write.content` or `edit.newString` JSON string containing a line that begins with a Python import, function definition or Triton decorator. It records the string's character span in the generated stream and the overlapping token indices, allowing each probability to be checked against the original response. The token count includes comments, whitespace and JSON escaping within the argument. All response request identifiers are distinct across the three directories, and three incomplete tool blocks are recorded separately.

Saved collection	Responses	Generated tokens	Responses with code arguments	Tokens in code arguments
`runs`	49	7,182	19	898
`runs-triton`	118	15,987	12	1,862
Migrated	28	3,360	2	402
Total	195	26,529	33	3,162

For example, response line 2 of `runs-triton/row_reduce-full-09/logprobs.jsonl` contains an `edit` argument defining `kernel_launch` with `tl.arange`, `tl.load`, `tl.sum` and `tl.store`. Response line 4 of `runs-migrated/row_reduce-full-00/logprobs.jsonl` contains a `write` argument beginning with `import triton` and including a `@triton.jit` function. Both decoded strings appear in their corresponding tool-event inputs.

The reconstruction preserves token boundaries while making complete code strings available for inspection. This matters when an identifier such as `triton` spans the generated fragments `tr` and `iton`. The audit stores the reconstructed arguments and their token-level evidence in `v2/local_tool_evidence/audit.json`; Appendix A gives the command used to regenerate it.

5. Discussion

5.1. Vocabulary attribution in the measured output

The main observations place the low-probability tail outside the vocabulary group highlighted by the domain-rarity explanation. CUDA/Triton terms have a lower threshold rate than other tokens, and the regional comparison places substantially more cost in surrounding prose. Contextual inspection further associates much of the tail with comments, names and malformed continuations.

Together, these results challenge the attribution of the measured probability cost to CUDA vocabulary. The selected kernel-writing tasks produce highly probable code continuations alongside a broader prose distribution, and their code-token minimum remains above the quoted floor. The distribution's dependence on output region provides a more direct description of the observed variation than the domain label alone.

5.2. The magnitude gap and its interpretation

The observed code-token minimum is approximately four hundred times the quoted floor, while the median code-token probability rounds to one. This gap motivates examining how model state, generated context and token selection enter the comparison. CUDA Agent discusses a base policy during rollout, while our main collections use an aligned model through an inference API [2]. Chlenski et al. examine cross-model fidelity and find that agreement in predictions can coexist with differences in attribution, making the measurement target and the kind of inference important to specify [14].

The empirical finding is that near-floor probabilities are absent from the measured code output, including its CUDA/Triton vocabulary. To connect this result to the training account, a matched base-model measurement would locate the same quantities before fine-tuning and show how their distributions change afterward. Such a comparison would directly test whether the quoted tail is associated with the base policy and how it relates to the vocabulary and regions identified here.

5.3. Regional distribution as the supported finding

The largest separation in the measured distribution occurs between code and prose. Under the fence rule, their threshold rates are 0.60% and 13.56%; removing comments lowers the code rate to 0.33%. This separation appears across all five operator-chain tasks, whose individual code-side rates remain low even when evaluation success is uncommon.

The contextual examples suggest an explanation for this pattern. Familiar code sequences, such as a masked `tl.load`, strongly constrain their continuations. Explanatory prose permits a wider range of wording and punctuation, distributing probability across more possible continuations. The concentration analysis reflects this difference: the lowest-probability 5% of code accounts for nearly all its cost, while prose accumulates cost over a much broader part of its token distribution.

The composition of the interaction also contributes to the aggregate result. Failure-following responses contain most of Family B's tokens and have a higher threshold rate than initial responses. Understanding the generated distribution therefore requires tracking both the kind of text produced and the evaluation context in which the model produces it.

5.4. A context-distribution account of the collapse

The regional measurements motivate a hypothesis linking training instability to the contexts generated during unsuccessful revision. CUDA Agent reports rising entropy alongside reward collapse [2], and DR. KERNEL finds that controlling train/inference mismatch stabilises training [6]. Our collections place much of the probability cost in prose produced around failed evaluations. These observations suggest examining the distribution of diagnostic and self-correction text encountered during rollout.

Under this hypothesis, failed evaluations lead the policy into contexts containing compiler diagnostics, revised explanations and attempts to recover from an unsuccessful implementation. Their continuation distribution may differ from that represented in successful demonstrations. Tracking how often training visits these contexts could help explain changes in entropy and importance ratios as trajectories lengthen.

Rejection fine-tuning offers a relevant comparison because it selects successful trajectories and thereby changes the context mixture used to anchor the policy [2]. In our Family B collection, success-following responses contain 12,147 tokens and failure-following responses contain 474,728. These counts illustrate the potential size of the mixture difference and motivate measuring its effect directly within a training run.

A second explanation places the uncertain prose downstream of an unresolved implementation problem. A struggling model may produce both an incorrect kernel and a diffuse explanation, with the prose recording the difficulty already present in the interaction. Distinguishing these explanations requires interventions that vary the generated context while holding other aspects of the task fixed.

Two comparisons would help. One could retain the content of a failure message while changing its diagnostic formatting, then measure the response distribution in code and prose separately. Another could request a complete rewrite after a successful evaluation, providing a comparison between rewriting after success and rewriting after failure. Logging the same regional probabilities, entropy and importance ratios during training would then connect these interaction-level measurements to the discrepancy examined by Jet-RL and the objective analysis of Zheng et al. [7, 8].

5.5. Classification bias in the first pass

The initial local-model analysis assigned a 10.00% low-probability rate to the `triton_specific` class, with a nominal significance value of 0.0156. Inspection revealed that the classifier's treatment of `load`, `zeros` and `store` depended on the region in which they appeared. To avoid labelling ordinary English as Triton vocabulary, the rule only admitted certain occurrences inside code segments. It consequently retained 20 of 451 relevant occurrences, including two extreme samples.

The resulting partition placed occurrences of the same word in groups with very different recorded rates:

Label assigned to `load`	Occurrences	Mean log-probability	Low-probability rate
`NL_reasoning`	6	−0.044	0.00%
`triton_specific`	4	−1.034	25.00%

Recomputing the comparison over all 451 occurrences changed the nominal significance value to 0.211, and the earlier vocabulary result was withdrawn. The error arose from selecting the comparison group through a region-dependent rule before calculating its statistics.

The revised analysis therefore evaluates vocabulary membership across the complete token stream and reports both strict and loose definitions. It also uses the original text to distinguish programming constructs from ordinary English and to examine what the low-probability token expresses. These procedures connect the reported rates to their underlying occurrences and make classification errors easier to identify.

5.6. Collection driver defect

The collection driver contributed substantially to Family B's failed evaluations. It accepted any fenced block as a candidate, including fragments intended as local revisions. In 14 of 39 runs examined for this defect, the model's last output contained a fragment of 63–239 characters; an example is `group_id = pid // (GROUP_M * num_pid_n)`. Executing such fragments as complete files produced 16 `NameError` failures among the 32 failed evaluations in that breakdown.

Four additional failures arose from out-of-memory errors after the task's peak memory use had been underestimated, and nine were output mismatches. Excluding fragment runs gives a success rate of 28% on complete files, or seven of 25. The driver defect also increases the number of failure-following responses in the collection, making it relevant to the context mixture measured in Section 4.6.

5.7. Complementary coverage of reply text and tool parameters

The two collection routes make code probabilities observable through different output interfaces. In Families A and B, the model prints a candidate in the reply body, and the driver performs file writing and evaluation. In the local collection, the model proposes a `write` or `edit` operation whose arguments contain the code. The generated stream retains those argument tokens and their probabilities, while the tool-event record retains the proposed operation and its execution status.

Reply-body collection enabled the main experiments through the tested DeepSeek API. The local records demonstrate probability capture through the agent's file-writing interface, including exact matches between recovered code strings and tool inputs. Together, these results provide practical collection paths for studying code generation both within replies and within tool operations.

6. Limitations

1. **Model state.** The main measurements describe an aligned model at inference. A matched measurement of the base policy before fine-tuning would clarify how the observed distribution relates to the rollout distribution discussed by CUDA Agent [2, 14].
2. **Model coverage.** Families A and B use one API model. The Qwen3-4B-AWQ records extend the collection interface to local tool calls and are reported as a separate supplement.
3. **Task coverage.** The selected operator chains combine a heavy operation with subsequent passes. Their implementation arrangements cover a limited part of the kernel-design space. More demanding choices involving shared-memory intermediates, access-pattern transformations, split-K reductions or online softmax would extend this coverage.
4. **Context composition.** Failure-following responses account for 87.5% of Family B's tokens. The reported regional distributions therefore weight unsuccessful revision heavily, and the driver defect contributes to that mixture.
5. **Statistical dependence.** Tokens within a completion are dependent. The nominal Fisher-test significance values use token-level counts; a run-level analysis would provide a complementary estimate of uncertainty.
6. **Training measurements.** The experiment records inference-time probabilities. Testing their relationship to importance-ratio instability requires collecting probabilities and training diagnostics within the same rollout and update process.
7. **Collection interface.** The main regional comparisons use reply-body code. The supplementary local audit documents code-probability capture inside tool arguments, providing a second interface for future comparisons.
8. **Region assignment.** Fence-based assignment includes comments, tracebacks and shell commands in the code region when they appear inside fences. The analysis quantifies the contribution of comments; finer annotation would separate the remaining forms of fenced content.
9. **Rewriting context.** A failed evaluation can change both the feedback content and the amount of new implementation the model must generate. Requesting a complete rewrite after success would help distinguish these contributions to the response distribution.

7. Conclusion

The measured probability cost is concentrated in the prose surrounding generated kernels, while CUDA/Triton vocabulary and code continuations have relatively low threshold rates. This pattern appears in collections containing 98,533 tokens on elementwise or reduction tasks and 542,611 tokens on operator chains. Contextual inspection locates much of the tail in comments, identifier choices, punctuation and malformed output.

Within the operator-chain collection, removing comments gives a code-side threshold rate of 0.33%, compared with 13.56% for prose. The lowest-probability 5% of code contributes 98.1% of its total cost, showing how sparse that tail is. The minimum code-token probability remains approximately four hundred times the quoted floor. These observations challenge the vocabulary-based attribution of the probability cost and direct attention to the text and contexts produced during kernel development.

The local audit additionally recovers probabilities for 3,162 tokens inside 40 code-bearing tool arguments. It establishes a collection route through file-writing tools alongside the reply-body method used in the main experiment. Together, the measurements and collection methods support further study of how implementation, explanation and evaluation feedback contribute to the probability distribution encountered by a kernel-writing agent.

Appendix A — Reproducibility

```
# Pre-flight
python scripts/check_plaintext_logprobs.py \
    --model deepseek-chat --max-tokens 16000

# Family A
python scripts/loop_driver.py --tasks row_reduce vector_affine \
    --conditions full truncated --repeats 16 --max-turns 8 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-formal
python scripts/loop_driver.py --heldout --tasks layernorm_rows softmax_rows \
    --conditions full truncated --repeats 16 --max-turns 8 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-formal

# Family B
python scripts/make_kb2_tasks.py --out tasks-kb2 --names \
    12_Gemm_Multiply_LeakyReLU 1_Conv2D_ReLU_BiasAdd 14_Gemm_Divide_Sum_Scaling \
    17_Conv2d_InstanceNorm_Divide 19_ConvTranspose2d_GELU_GroupNorm
python scripts/loop_driver.py --tasks-root tasks-kb2 \
    --tasks 12_gemm_multiply_leakyrelu 1_conv2d_relu_biasadd \
            14_gemm_divide_sum_scaling 17_conv2d_instancenorm_divide \
            19_convtranspose2d_gelu_groupnorm \
    --conditions full truncated --repeats 4 --max-turns 6 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-kb2

# Analysis
python scripts/stream_stats.py lowprob-attribution/runs-kb2
python lowprob-attribution/scripts/make_v2_figures.py

# Audit saved local tool-call code and its probabilities (from repository root)
python lowprob-attribution/scripts/audit_local_tool_logprobs.py
```

Appendix B — Production record

The study was produced with an AI assistant. `PROCESS.md` records the collaboration history, including the withdrawn classification result and its correction. The v2 experiment extends the initial reduction-task collection with KernelBench Level 2 operator chains, and the supplementary local audit records the saved tool-argument evidence.

References

[1] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-Turn RL for Generating CUDA Kernels, 2025. URL https://arxiv.org/abs/2507.11948v1. arXiv:2507.11948v1.

[2] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation, 2026. URL https://arxiv.org/abs/2602.24286v1. arXiv:2602.24286v1.

[3] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels?, 2025. URL https://arxiv.org/abs/2502.10517v1. arXiv:2502.10517v1.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017. URL https://arxiv.org/abs/1707.06347v2. arXiv:1707.06347v2.

[5] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation, 2015. URL https://arxiv.org/abs/1506.02438. arXiv:1506.02438.

[6] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. Kernel: Reinforcement Learning Done Right for Triton Kernel Generations, 2026. URL https://arxiv.org/abs/2602.05885v2. arXiv:2602.05885v2.

[7] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, and Ligeng Zhu. Jet-RL: Enabling On-Policy FP8 Reinforcement Learning with Unified Training and Rollout Precision Flow, 2026. URL https://arxiv.org/abs/2601.14243v2. arXiv:2601.14243v2.

[8] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing Reinforcement Learning with LLMs: Formulation and Practices, 2025. URL https://arxiv.org/abs/2512.01374v3. arXiv:2512.01374v3.

[9] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your Efficient RL Framework Secretly Brings You Off-Policy RL Training. Technical report, August 2025. URL https://fengyao.notion.site/off-policy-rl. Technical report, first published 5 August 2025.

[10] Yingru Li, Jiacai Liu, Jiawei Xu, Yuxuan Tong, Ziniu Li, Qian Liu, and Baoxiang Wang. Trust Region Masking for Long-Horizon LLM Reinforcement Learning, 2025. URL https://arxiv.org/abs/2512.23075v5. arXiv:2512.23075v5.

[11] Timothée Chauvin, Erwan Le Merrer, François Taïani, and Gilles Tredan. Log Probability Tracking of LLM APIs, 2026. URL https://arxiv.org/abs/2512.03816v2. ICLR 2026; arXiv:2512.03816v2.

[12] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A. Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, Itay Yona, Eric Wallace, David Rolnick, and Florian Tramèr. Stealing Part of a Production Language Model, 2024. URL https://arxiv.org/abs/2403.06634v2. arXiv:2403.06634v2.

[13] James A. Michaelov, Catherine Arnett, Tyler A. Chang, Pamela D. Rivière, Samuel M. Taylor, Cameron R. Jones, Sean Trott, Roger P. Levy, Benjamin K. Bergen, and Micah Altman. How Open Must Language Models be to Enable Reliable Scientific Inference?, 2026. URL https://arxiv.org/abs/2603.26539v2. arXiv:2603.26539v2.

[14] Philippe Chlenski, Zachariah Carmichael, Ayush Warikoo, Chia-Tse Shao, Yingxiao Ye, Aobo Yang, Vivek Miglani, and Nehal Bandi. Surrogate Fidelity: When Can Open LLMs Explain Closed Ones?, 2026. URL https://arxiv.org/abs/2606.32008v1. arXiv:2606.32008v1.